

PROJECT AND TEAM INFORMATION

Project Title

MemOpt(Tiered Memory Management beyond hotness)

Student / Team Information

<i>Team Name:</i> TierX <i>Team #</i>	
Team member 1 (Team Lead)	<i>Bhargava, Garima –</i> <i>240121232</i> <i>garimabhargava4@gmail.com</i> 
Team member 2	<i>Bhatt, Nandini – 24022559</i> <i>nandinibhatt006@gmail.com</i> 
Team member 3	<i>Tomar, Shaili – 240221680</i> <i>shailitomar0112@gmail.com</i> 
Team member 4	<i>Bisht, Saumya –</i> <i>240221898</i> <i>saumyabisht179@gmail.com</i> 

PROPOSAL DESCRIPTION

Motivation

Memory management is an important part of an operating system because it affects computer performance. Modern systems use different types of memory such as fast **DRAM** and slower **remote or CXL memory**. Fast memory is better but it is limited and costly so it is important to use it well.

Most memory management techniques decide which data should be placed in fast memory by checking how often it is accessed. Frequently accessed data does not always have the biggest effect on system performance. Some data may be accessed less often but can cause higher **memory latency** and **CPU waiting time**. Using only access frequency can also waste memory bandwidth by moving data again and again.

The goal of **MemOpt** is to improve memory placement by looking at the effect of memory accesses on performance instead of only their frequency. The system will use **Amortized Offcore Latency (AOL)** which combines memory latency and **Memory Level Parallelism MLP** to find data that is more important for performance.

Based on this information **data that is important for performance** can get priority in fast memory while unneeded data movement can be reduced.

The main aim of this project is to make better use of memory, reduce **unneeded page migration** and improve **overall system performance**.

DBMS Connection: Database systems use memory to store table data, indexes and database pages. These pages can have different effects on query performance. In this project, a DBMS such as PostgreSQL will be used with MemOpt to study how database data uses memory and to improve its placement between fast and slow memory.

State of the Art Current Solution

Today operating systems manage fast DRAM and slower CXL memory using a simple rule which is to move data based on **how often it gets accessed**.

Popular techniques such as **Linux TPP** and **NUMA balancing** check how often a memory page is used. If a page is used often the system marks it as **hot** and may move it from slower memory to fast DRAM.

The problem with this approach is that it treats every access in the same way. It assumes that frequently used data has the biggest effect on system performance. As a result, current systems may waste time and memory bandwidth by moving pages between memory tiers even when those pages do not affect CPU performance much.

Today's solutions mainly focus on **access frequency** instead of the **actual performance impact** of memory accesses.

Current memory management methods do not consider DBMS information such as whether a memory page belongs to a table or an index and how important it is for a query. Our approach will combine DBMS information with memory performance information such as AOL to make better placement decisions.

Project Goals and Milestones

The main goal of **MemOpt** is to improve memory management in **tiered memory systems**. Current techniques place data in fast memory mainly based on how often it is accessed. High access frequency does not always hurt system performance

MemOpt solves this issue by finding **data that is important for performance** based on its effect on execution time and by looking at **memory access latency** and **Memory Level Parallelism MLP**

Project Goals

- Study existing **memory tiering techniques** and understand the limits of frequency based data placement
- Capture real time memory access patterns using **hardware profiling tools** such as Linux perf
- Measure **access latency** and **MLP** to calculate the **Amortized Offcore Latency AOL** score
- Build a placement method that moves **high AOL data** into fast memory while reducing unneeded page migrations
- Compare MemOpt performance with traditional **hotness based methods** using memory intensive workloads
- Integrate a DBMS such as PostgreSQL with MemOpt for database workloads
- Use database page and query information along with AOL for better memory placement

Milestones

1. Complete the basic study on **Linux memory tiering** and set up the profiling environment
2. Implement and check the **AOL calculation method** using live hardware performance counters
3. Develop the **MemOpt modules** for better data placement and page migration control.
4. Test the system with suitable workloads and compare the results with traditional methods and complete the **final project report**
5. Integrate PostgreSQL and study its database memory usage before testing MemOpt.

Project Approach

MemOpt will use a **step by step approach** to improve memory placement in a tiered memory system

First existing **memory management techniques** and hotness based approaches will be studied to understand how they work and their limits

Next memory performance information will be collected using available **profiling tools and hardware counters**. The data will be used to study **memory latency** and **Memory Level Parallelism MLP**

These values will be used to calculate the **Amortized Offcore Latency AOL** score to find data that has a greater effect on system performance

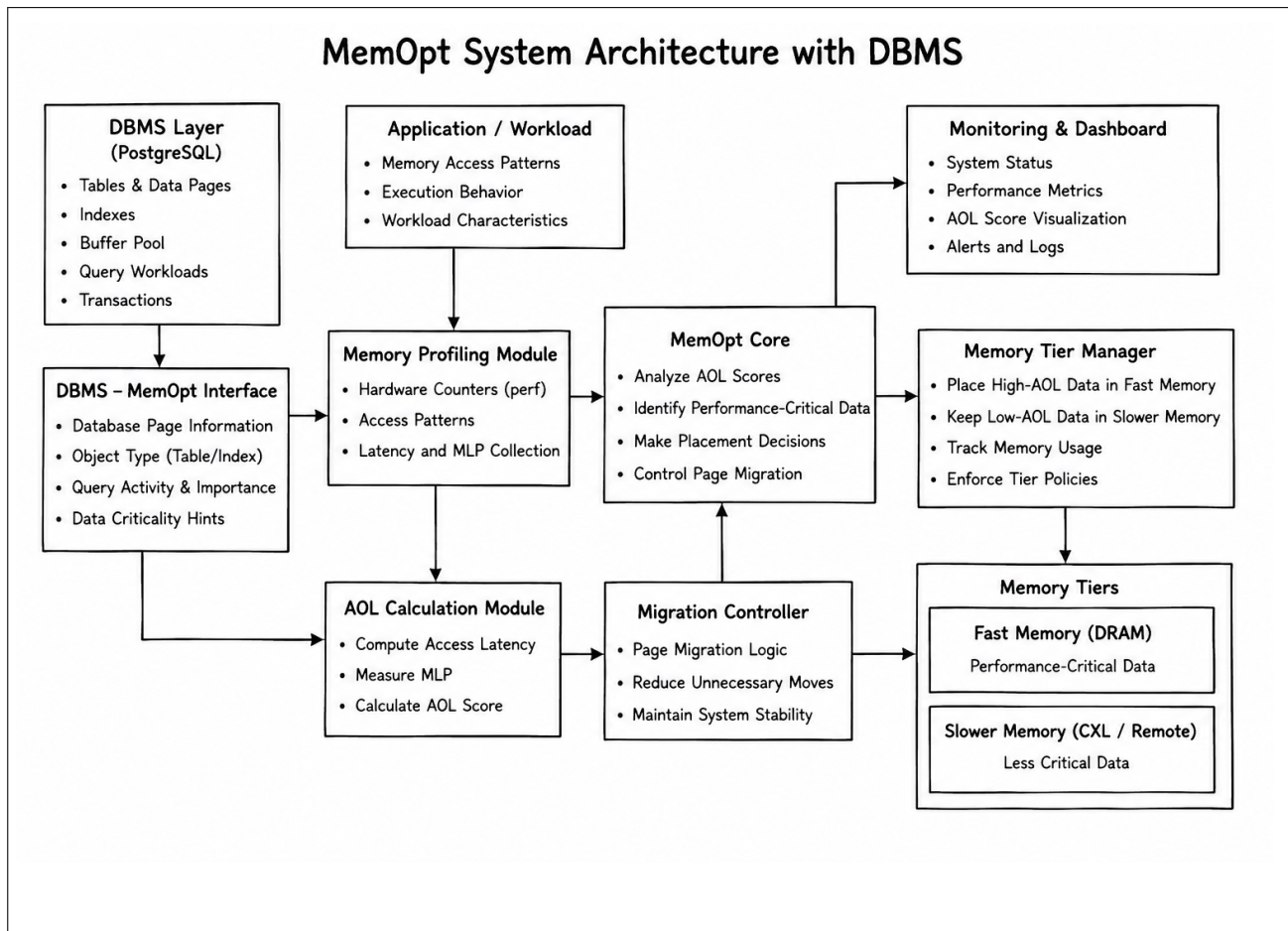
The system will then use this information to make better **memory placement decisions**. Data that is important for performance will get priority in fast memory while unneeded movement of less important data between memory tiers will be reduced

Finally MemOpt will be tested using suitable workloads. Results such as **execution time CPU waiting memory performance and page migration activity** will be recorded and compared with traditional hotness based approaches

This test will show if the proposed approach improves the **efficient use of fast memory** and overall system performance

A DBMS such as PostgreSQL will be used to generate database workloads. Information about database pages, tables, indexes and query activity will be considered along with memory performance data.

System Architecture



Project Outcome Deliverables

The main outcome of **MemOpt** will be a prototype memory system made to improve data placement across different memory tiers

The system will include a **memory profiling module** to collect information about memory accesses latency and **Memory Level Parallelism MLP**. MemOpt will use this information to calculate the **Amortized Offcore Latency AOL** score and find data that has a greater effect on CPU performance

Another outcome will be a method for placing **data that is important for performance in fast memory** while reducing unneeded page migrations between memory tiers

The proposed approach will be tested using **memory intensive workloads** and compared with traditional hotness based memory management techniques.

The project will also demonstrate DBMS-aware memory management using PostgreSQL database workloads. MemOpt will use database information along with AOL to identify important database data and improve its placement across different memory tiers.

Final Deliverables

- **MemOpt source code**
- **System architecture and design documentation**
- **Benchmark results**
- **Performance analysis**
- **Final project report**
- **DBMS integration with MemOpt**
- **Database workload benchmark results**

Assumptions

The project uses the following assumptions

- The required **hardware and software tools** are available to collect memory performance data.
- Both **fast memory DRAM** and slower memory are available for testing
- The selected programs or workloads use enough memory to show performance differences
- The collected data is accurate enough to measure **memory latency MLP and AOL**
- A **DBMS** such as **PostgreSQL** is available for generating database workloads

References

- **Jinshu Liu et al.**, "Tiered Memory Management Beyond Hotness," **OSDI 2025**.
- **Linux Kernel Documentation** – Memory Management and Memory Tiering.
- **Intel Documentation** – Performance Monitoring Units (PMU) and PEBS.
- **Linux perf Documentation** – Hardware Performance Monitoring.
- **PostgreSQL Documentation** – Database System and Memory Management